PRODUCT BRIEF

UBB — Product Brief

What UBB is, what has been built, what has been verified, and what remains before a real launch

10 November 2026

Product brief derived from the verified R10-ASH-HANDOFF-2 engineering handoff. Not production or rail-activation evidence.

Audience: the owner, advisers, the operational lead, prospective customers and partners. Every capability is labelled BUILT, LOCALLY VERIFIED, RELEASED, or NOT YET DEPLOYED / EXTERNAL WORK.

Contents

1. Executive summary 3

2. The product in one picture 5

3. What UBB actually does 5

4. Billing modes 8

5. Plans, subscriptions and referrals 9

6. Integration surfaces 10

7. The UI and console 11

8. Operator and admin workflows 13

9. What was actually verified 13

10. What is NOT built, and what is built but not yet activated 14

11. What remains before a real launch 15

12. Who this could be sold to, and initial positioning 16

13. A page for the founder: what have we built? 18

14. Appendix — exact release identities 19

What UBB is, what has been built, what has been verified, and what remains before a real launch.

How to read the labels in this document. Every capability is tagged with one of four honest states:

Label	Meaning
BUILT	The feature exists in the codebase and has automated tests.
LOCALLY VERIFIED	It was also exercised end to end on a developer machine in November 2026, with real database, queue and (where relevant) real Stripe data objects, but with no connection to Stripe and no public internet.
RELEASED	A versioned artefact exists that can be handed to a client.
NOT YET DEPLOYED / EXTERNAL WORK	It depends on hosting, real Stripe accounts, public internet endpoints or human sign-off that has not happened yet.

Nothing in this document is evidence of a production deployment, of real payments flowing, or of a real client being live. Where a sentence is an interpretation rather than a verified fact, it says so.

1. Executive summary

What is UBB?

UBB is a usage-metering, pricing, spend-control and margin service that sits between a software business and Stripe. A business that sells access to AI models or other metered APIs plugs UBB into its application. Every time a user of that business does something that costs money, the application tells UBB once. UBB works out what that action cost the business (the supplier cost), what the business charges its own customer for it (the customer price), records both permanently with a receipt explaining how the numbers were reached, checks whether any spending limit has been crossed, and answers immediately with the cost, the price and a clear "stop" or "carry on" verdict.

Stripe remains the billing engine: it sends invoices, collects payment, handles tax, chases late payers, processes refunds and disputes. UBB is the control plane on top of it that decides *what* to bill, keeps the economic truth, and stops spending before it runs away.

Who is it for?

Businesses whose product consumes something metered and costly, usually AI model calls, and who resell that consumption to their own customers. In UBB's vocabulary that business is a **tenant**, and the people it sells to are its **customers**.

What problem does it solve?

Such businesses face four problems at once. They need to know their true cost per customer when suppliers price by token, request or second. They need to charge customers in a way that preserves margin. They need to stop a customer, an agent or a runaway job before it spends money the business cannot recover. And they need all of this to be exact, auditable and replay-safe, because money is involved. UBB is built to solve those four problems as one system rather than four scripts.

What can a client do with it today?

A client can create its customers in UBB, declare the kinds of usage it meters, set supplier costs and customer prices, send usage events from its code, group work into tasks with spending ceilings, receive stop signals in real time, read per-customer revenue, cost and margin, let its own end customers read their usage, and receive webhooks when things happen. It can do this through a versioned REST API, a released Python SDK (version 5.0.0), or code generated for it by UBB's Code Builder. A browser-based tenant console exists for the client's team. All of this is **BUILT** and **LOCALLY VERIFIED**.

Current readiness state

- The billing engine has been through a full internal repair-and-verification programme. It carries **zero known billing blockers** and **zero known pre-handoff required fixes**.
- The Python SDK 5.0.0 is **RELEASED** as a tagged, reproducibly built, independently verified package.
- UBB has **never been deployed** to any server. No real Stripe account, public webhook endpoint, production login environment, hosting or service-level measurement exists yet. Those are the next body of work and are described in section 11.
- The first client is planned to launch in **external** billing mode, in which UBB meters, prices, controls and reports but moves no money. Money modes (prepaid and postpaid) are built and locally verified but need real Stripe readiness and a deliberate human activation step before use.

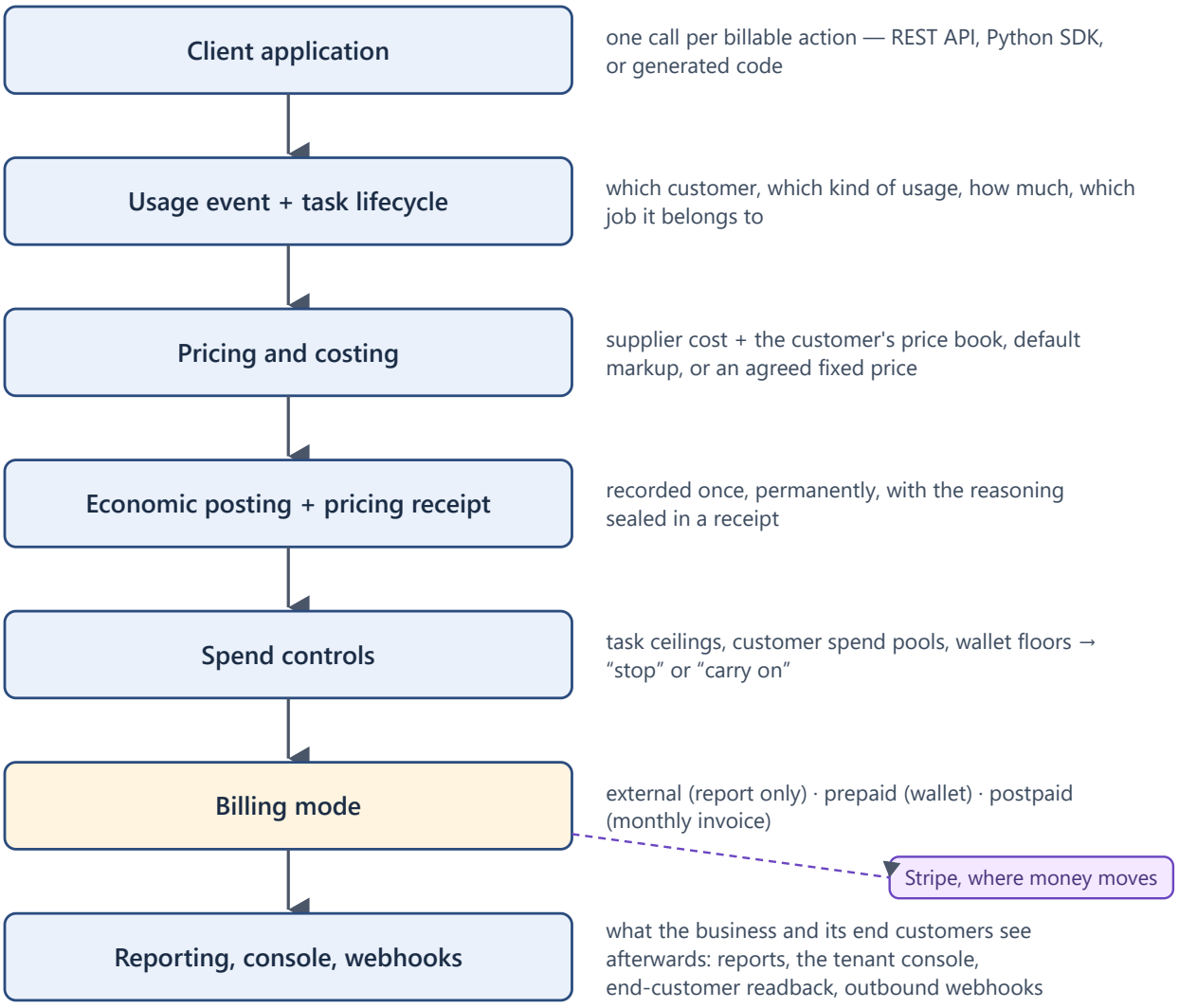
This document does not describe UBB as production-ready. It describes a verified product awaiting its operational launch work.

One-paragraph product description (for the owner to adapt)

> *UBB is the usage, spend-control and margin layer for businesses that resell AI and API consumption. It meters every billable action once, prices it against the business's own supplier costs and customer price books, records an immutable economic receipt, enforces spending ceilings and customer balances in real time, and reports true per-customer revenue, cost and margin. It drives Stripe for invoicing and collection rather than replacing it, supports prepaid wallets, postpaid invoicing and seat-based subscription plans, and integrates through a versioned REST API, a Python SDK and generated integration code.*

2. The product in one picture

The flow below is explanatory, not an implementation diagram.



Reading it from the top: the client's software makes one call per billable action. UBB attaches that action to a customer, a declared kind of usage and, optionally, a task. It prices the action, writes the economic record with its receipt, applies every relevant spending control, and answers. Depending on the tenant's billing mode, the priced usage either stays as a report, draws down a prepaid wallet, or accrues towards a monthly Stripe invoice. Everything is visible afterwards through reports, the console, the end-customer readback API and outbound webhooks.

3. What UBB actually does

All capabilities in this section are **BUILT** and **LOCALLY VERIFIED** unless stated otherwise.

Usage metering

The client sends one HTTP call per billable action, or a batch of up to a hundred. Each call carries an idempotency key chosen by the client. If the same key is sent twice, UBB returns the original answer instead of recording the action again. A recorded action is answered with the supplier cost, the customer price, how each was determined, the task's running totals, and the stop verdict. There is no delayed or "best effort" ingest path: either the call is recorded and answered, or it is refused and the client knows.

Why it matters. Metering is the foundation of everything else. If an action is recorded twice, the customer is overcharged and the margin figures lie. UBB's replay rule and single-record design remove that class of error.

Event Types

Before usage can be priced, the tenant declares what it meters. An Event Type names a kind of action (for example "chat completion from supplier X"), the measurements it carries (tokens, seconds, requests), and how its supplier cost is known: either calculated from a cost book, or reported by the client's code at the time of the call. Event Types are drafted and then published; publication pins the declaration so later changes cannot silently re-price history. If the client sends a measurement name nobody has declared, UBB records the event, marks its cost and price as unresolved, and holds the unknown name for a later publication or resolution run. Nothing is dropped and nothing is guessed.

Why it matters. AI products change fast. Declared Event Types let the business add models and suppliers without rewriting its billing, while keeping an honest record of anything it has not yet classified.

Supplier cost and customer pricing

UBB keeps **cost books** (what suppliers charge the tenant) and **pricing books** (what the tenant charges its customers). A pricing book can be the tenant's default, attached to a plan, or an override for one customer; the most specific book that applies wins, and there is no silent fall-through between books. If no rule matches, a tenant-wide default markup applies. Prices change through dated draft-and-publish changes, so there is always an answer to "what was the price on that day". Fixed-price work (an agreed price for a whole task) is also supported.

Immutable pricing and economic receipts

Every recorded action becomes a **posting**: a permanent economic record with the supplier cost, the customer price, the status of each (known, unresolved, waived, unknown or not applicable) and a **pricing receipt**. The receipt is a structured explanation of how the numbers were reached: which book, which rule, which markup, which engine version. Once complete, a receipt is sealed by the database and cannot be edited. Money is stored as whole micro-dollars (millionths of a US dollar) in integer arithmetic, so there is no floating-point drift.

Why it matters. When a customer disputes a bill, or an investor asks about gross margin, the business can show exactly how every figure was produced. This is the difference between a dashboard and an audit trail.

Tasks and Subtasks

A **Task** is a unit of work the client defines: an agent run, a document pipeline, a batch job. Usage events can name the task they belong to, and the task accumulates their cost and price. A task can have a spending ceiling, a silence window (how long it may go quiet before it is treated as abandoned) and an absolute deadline. Crossing the ceiling kills the task and anything beneath it; the kill is recorded with its reason and announced by webhook. Subtasks nest one level under a task. A task can also carry an agreed fixed price that is charged when the work is delivered.

Why it matters. AI agents can loop and spend without limit. Tasks give the business a place to say "this job may cost at most this much" and have the platform enforce it.

Spend controls

Four families of control exist:

Control	What it does
Task ceilings	A cost cap per task or task type; crossing it kills the work.
Admission control	A rate limit on how many tasks a tenant may start per minute.
Customer spend pools	A spending cap per customer over a period, in alert-only or blocking mode, with threshold alerts.
Wallet policy	In prepaid mode, a soft floor that refuses new top-level work and a hard floor that suspends the customer.

A tenant-wide switch selects **off** (decisions are made from durable database truth, with hourly reconciliation) or **enforcing** (a real-time live counter and a stop-signal ledger stop work within the same request). The client always learns of a stop in three ways: in the response to the call that crossed the line, as a refusal when it tries to start new work, and as a webhook.

Customer economics and margin

UBB reports supplier cost, customer revenue and gross margin per customer, per period and by any grouping field the tenant declares (model, feature, region, team). Each figure carries a status: known, incomplete, or unavailable at that grain or beyond the retention horizon. Revenue and cost are read from postings only. Wallet credits, grants, refunds and invoice pushes never move them, which was explicitly proven in the local verification.

Idempotency and economic truth

The same discipline runs through every money operation: credits, debits, refunds, top-ups, invoice pushes, Stripe refund claw-backs and subscription seat changes all carry an idempotency key, and a replay returns the original result rather than acting twice. Database constraints and triggers back the rules in code, so even a bug cannot silently double-record or un-seal a receipt.

Reporting

Beyond economics, the tenant can read stops and breaches, utilisation and headroom against controls, task analytics, billing periods and invoices, and a full audit log of administrative actions. End customers can read their own usage, revenue basis, balance, grants, transactions and invoices through a short-lived **widget token** the tenant issues; a tenant API key is refused on that surface. *(Interpretation: there is no embeddable UI widget shipped with UBB; the end-customer surface is an API the tenant renders itself.)*

Webhooks

A tenant registers HTTPS endpoints and chooses which of 38 event types it wants: usage recorded, task killed or expired, customer stopped or suspended or reactivated, wallet balance low, top-up requested, usage invoice pushed, referral reward earned, and so on. Deliveries are signed, retried with backoff on transient failure, and recorded. Only public HTTPS addresses are accepted; private and loopback addresses are refused at registration and again at send time. Signing secrets can be rotated with an overlap window.

4. Billing modes

A tenant runs in exactly one of three billing modes. The first client launches in **external**. The two money modes are **BUILT** and **LOCALLY VERIFIED** but are **NOT YET PRODUCTION-ACTIVATED**: each requires a real Stripe connected account and a deliberate, human-approved **payment-rail activation** record for the environment, which does not yet exist outside disposable test databases.

External — meter, price, control and report; move no money

UBB does everything in section 3 but holds no wallet, sends no invoice and touches no payment rail. Every wallet operation, including read-only ones, is refused with a clear "feature not enabled" answer rather than returning misleading zeros. A tenant may still enable the billing product in this mode to use plans, spend pools and postpaid configuration ahead of a later move to a money mode.

This is the first-client launch mode. It lets a real business use UBB's metering, pricing, spend control and economics while invoicing its customers by whatever means it already uses.

Prepaid — a wallet per customer

Each customer (or, for pooled seats, the business that owns them) holds a wallet denominated in micro-dollars.

- **Credits and grants.** Money enters by an operator credit, a Stripe top-up, or a **credit grant** (paid or promotional). Grants carry an expiry and are consumed before paid balance.
- **Top-ups and auto-top-up.** A customer can top up through Stripe Checkout on the tenant's connected account. The tenant can configure automatic top-up when the balance falls below a threshold. A top-up that cannot proceed (Stripe not configured, account not charge-ready, rail closed) is refused before anything is written, with the specific reason.
- **Drawdowns.** Every priced usage event draws the wallet down exactly once, keyed on the event.
- **Affordability.** Before a task starts, UBB checks whether the wallet can afford it, and can **reserve** an agreed fixed price for the duration of the work, releasing it when the work ends.
- **Soft and hard floors.** A soft floor refuses new top-level work. A hard floor suspends the customer. In **off** mode suspension is cleared by the next credit or the hourly pass once the balance recovers; in **enforcing** mode the live counter stops work in real time and the same recovery clears the stop. Reactivation emits exactly one "customer reactivated" event.
- **Refunds.** A usage refund credits the wallet and returns one durable refund identifier on the first call and every replay. Stripe refunds and disputes claw back exactly the incremental amount, never twice. If Stripe later reports a refund as failed, UBB raises an alert for a human rather than silently re-crediting.
- **Withdrawals and manual holds.** Withdrawals are ledger lines on the same wallet. An operator can place or release a manual hold on a customer; releasing it never forces a customer active past a standing money cause.

Postpaid — usage accrues, Stripe invoices monthly

There is no wallet. Each billing owner's month of priced usage is pushed to Stripe as one invoice with line items, or ridden onto the renewal invoice of an active subscription.

- **Usage invoice periods** accrue during the month and close afterwards.
- **Stripe invoice push** is resumable and idempotency-keyed, bounded by attempt and age caps, and can be re-pushed or re-billed by an operator. A month whose total does not reach a whole cent is recorded as skipped, with the residue carried forward exactly once rather than invoiced as zero.

- **Payment status** follows Stripe's paid, payment-failed and voided events. A failed payment suspends the customer who owned the invoice at the time it was created; the final settlement reactivates them, once. If a customer is also past a wallet floor or inside a manual hold, a payment recovery hands over to that cause rather than wrongly reactivating.
- **Billing-owner semantics.** Each invoice remembers who owed it. A pooled seat's usage is owed by its business; an allocated seat is its own billing owner. The owner is frozen on the invoice by a database trigger, so later reshuffles of seats cannot re-address old debts.
- **UBB's own platform fee** to the tenant is a separate Stripe invoice on UBB's account, with the tenant's Stripe customer set by an operator command.

Comparison

	External	Prepaid	Postpaid
Meters, prices, receipts, economics	Yes	Yes	Yes
Task ceilings, spend pools, admission control	Yes	Yes	Yes
Customer wallet	No	Yes	No
Top-ups and auto-top-up via Stripe	No	Yes	No
Soft / hard balance floors	No	Yes	No
Monthly Stripe usage invoice	No	No	Yes
Payment-failure suspension and recovery	No	No (wallet floors instead)	Yes
Requires Stripe connected account	No	Yes	Yes
Requires payment-rail activation	No	Yes	Yes
Readiness	BUILT, LOCALLY VERIFIED, planned first-client mode	BUILT, LOCALLY VERIFIED, not production-activated	BUILT, LOCALLY VERIFIED, not production-activated

5. Plans, subscriptions and referrals

Plans

A **plan** is a catalogue entry with up to three pricing axes: a recurring access fee, a per-seat fee, and a usage markup carried by an attached pricing book. Plans are created, edited and archived by the tenant, and a plan can be assigned to a customer. The usage axis is applied at pricing time by the metering engine; the fee axes are realised as Stripe Prices when a subscription is created. Plans are **BUILT** and **LOCALLY VERIFIED** without Stripe for everything up to and including usage-only assignment.

Subscriptions

When a plan carries fees, subscribing a customer creates a Stripe subscription on the tenant's connected account. UBB keeps a **mirror** of that subscription so it can answer questions without calling Stripe, refreshed by Stripe webhooks and an hourly reconciliation. Seat counts are pushed to Stripe when the customer's roster changes, and an hourly reconciler converges any missed push. Pause, resume, cancel and re-price are driven by UBB as calls into Stripe. Subscribing is one all-or-nothing outcome: if Stripe refuses, no plan assignment, no audit entry and no mirror is left behind. A mirror binds to its customer once and never rebinds; an ambiguous match binds nowhere and raises an alert. Subscription invoices remember their billing owner in the same frozen way as usage invoices.

Everything above is **BUILT** and **LOCALLY VERIFIED** using real Stripe data objects with the network blocked. Operating it against a real Stripe subscription on a real connected account is **EXTERNAL WORK**.

Referrals

A separately enabled referrals product lets the tenant run **programs** that reward a referrer with either a flat fee (in micro-dollars, paid once per referral) or a share of the referred customer's spend (in basis points, exact to the unit). Referrers and codes are managed, attribution is recorded, rewards accumulate in a per-period ledger reconciled daily, and payouts can be exported. Referral events are available as webhooks. Referrals do not need a payment rail. **BUILT** and **LOCALLY VERIFIED**; not enabled for the first client.

Which of these need real Stripe readiness

Capability	Proven locally	Needs real Stripe / rail before use
Plan create / edit / archive / assign (usage-only)	Yes	No
Fee-bearing subscribe, seats, pause / resume / cancel, re-price	Yes, with real Stripe objects and no network	Yes
Subscription mirror, sync, seat reconciler, webhook handling	Yes	Yes, for live traffic
Referral programs, attribution, rewards, payout export	Yes	No

6. Integration surfaces

REST API

One versioned contract at `/api/v1`, published as an OpenAPI document that is the single source of truth for the client surface. Clients authenticate with a tenant API key. A key prefixed for test routes to a **sandbox** sibling of the tenant that can be reset on demand, so clients can integrate safely before touching real data. The contract is protected by automated gates that refuse breaking changes unless deliberately recorded. **BUILT**.

Python SDK

The SDK is **RELEASED** as version 5.0.0:

Item	Value
Release tag	`sdk-v5.0.0`
Package	`ubb_sdk-5.0.0-py3-none-any.whl`
How it is delivered	Handed over by file. It is **not** published to PyPI or any public index.
Verification	Built twice with identical results from the tagged source, installed into a clean environment, and the full SDK test suite run against the installed package.

The SDK wraps a generated core that is stamped with the fingerprint of the exact API contract it was built from, so a mismatch between SDK and server is detectable. Version 5 is the one deliberate breaking release after the pre-launch cutover; it is the version a first client receives if it integrates in Python.

Code Builder

In the console's Developers area, a tenant selects what it wants to integrate: which customers, Event Types, tasks and behaviours. UBB turns that selection into an **Integration Blueprint** and renders it into a ready-to-run Python module (targeting SDK major version 5) or a shell script. The generated code is exercised by an automated suite that renders and runs integrations inside containers, so what the builder produces is proven to execute. **BUILT** and **LOCALLY VERIFIED**.

Webhooks

Two directions exist. **Outbound**, the tenant's own webhooks described in section 3. **Inbound**, two Stripe receivers: one for billing events (payments, refunds, disputes, usage invoices) and one for subscription events. Each verifies Stripe's signature with its own secret, refuses to run if that secret is missing, and de-duplicates independently, so a Stripe event sent to both is handled once by its owner. A failed event can be re-run by an operator from its own record. Receiving real Stripe events requires public HTTPS endpoints that do not yet exist: **EXTERNAL WORK**.

7. The UI and console

UBB does include a browser-based tenant console, built in React, for the client's own team. It is a tenant console, not a public end-customer portal and not an operator back-office. It can run against a mock data provider for demonstration, or against the real API with a Clerk sign-in.

What is implemented today

Area	What the client's team can do there
Overview	A landing dashboard for the workspace.
Events	Browse recorded usage events and open one to see its receipt and measurements.
Tasks	See task kinds, runs and run detail, including stop facts and totals.

Area	What the client's team can do there
Customers	List and create customers; open a customer for usage charts, revenue and margin trend, a billing tab (balance, credits, debits, top-up, withdrawal, affordability, transactions, grants, billing profile with floors and auto-top-up), a customer spend pool section, usage invoices, and a subscription tab (subscribe, seats, pause, resume, cancel).
Pricing	Pricing books, book detail and dated book changes.
Billing (billing product)	Revenue section and chart, a credit and debit form keyed by the customer's platform identifier, the customer spend pool card, postpaid configuration, usage invoices.
Plans (billing product)	List, create and edit plans.
Referrals (referrals product)	Programs, referrers and referrer detail.
Spend controls	Configure the control families and read stops, breaches and headroom.
Webhooks	Register endpoints, choose event types, view deliveries and webhook detail.
Developers	API keys, the sandbox and its reset, and the Code Builder page with its generated-file panel.
Settings	Workspace settings, team members, enabled products, the tenant's own billing (UBB's platform-fee invoices with their Stripe references), and the audit log.

Wallet-related panels appear only where the tenant's billing mode actually holds a wallet, so an external-mode tenant is not shown misleading balance controls.

Verification status: the console's automated test suite, type checks and production build all passed in the local verification, and its generated API client matched the published contract exactly. **BUILT** and **LOCALLY VERIFIED** in mock mode and by automated tests.

What the UI is NOT yet

- **It has no production deployment.** Nobody outside the development machine has used it.
- **The production sign-in environment does not exist yet.** Member login uses Clerk; the production Clerk environment and its keys are external work. The server-side dependency for verifying those logins was repaired and proven in a clean environment, so the remaining gap is the environment itself.
- **Real-browser use against the live API is unproven.** The console was verified by automated tests and in mock mode. Running it in a browser against the real API end to end is external work.
- **Some operations remain API or operator workflows rather than console features.** Known examples: assigning a plan to a customer has no dedicated console workflow (the console shows plan information but assignment is an API call); Stripe Connect onboarding is driven through the API; tenant creation, the first API key, payment-rail activation, manual holds and platform-fee customer setup are operator-side (section 8). *(Interpretation from the code inventory, not a verified list of every gap.)*
- **It is not a finished self-serve SaaS product.** The source evidence supports calling it a working, tested tenant console covering the product's main surfaces, and no more than that.

8. Operator and admin workflows

Some actions are deliberately kept out of the client-facing API and console and placed with UBB's own operators. This is governance by design, so that the actions that create tenants, open money movement or override a customer's state always have an accountable human behind them.

Action	How it is done	Why it is operator-side
Creating a tenant and minting its first API key	Operator-run management shell using the platform's own service code; the key is shown once	A tenant is a commercial relationship, not a self-serve sign-up, for the first launch
Payment-rail activation	An operator-approved activation record per environment, naming the approver and the evidence	Opening real money movement is an accountable human act
Manual hold on a customer	One operator command that places or releases the hold and writes an audit entry	A hold overrides automated state and must be traceable
Setting the Stripe customer UBB bills for its platform fee	One operator command, optionally verified against Stripe, audited	It binds a tenant to a real payment relationship
Re-running a failed Stripe webhook; re-pushing a usage invoice	Operator commands acting on the original record	Recovery actions must act on the same record, never create a parallel one

Every one of these writes to the audit ledger. The only operator UI is the standard Django admin; there is no bespoke operator console.

9. What was actually verified

In November 2026 the billing product went through a dedicated verification pass on a developer machine, after a repair programme that fixed every issue an earlier review had found. The pass used a real PostgreSQL database, a real Redis instance, the real background worker and scheduler, and, where Stripe was involved, real Stripe data objects constructed in-process with all network transport blocked and no real key present.

The local verification covered:

- **External mode:** the full first-client path from configuration to recorded usage, task lifecycle, end-customer readback and webhook registration, with the confirmation that no wallet exists.
- **Prepaid under both enforcement modes:** credits, grants, drawdown exactly once, soft and hard floors, suspension and the single reactivation, manual hold, top-up refusals with the right reasons, refunds and their replay, Stripe refund and dispute claw-backs, reservations.
- **Postpaid:** accrual, period close, the invoice push with real Stripe invoice objects, sub-cent skips and carry, failure and re-push, payment-failed suspension and recovery, the platform-fee invoice.
- **Plans and subscriptions:** the all-or-nothing subscribe, seats and the converging reconciler, pause, resume, cancel, re-price, mirror sync and bind-once, both Stripe receivers over the wire with their own secrets.
- **Referrals:** basis-point and flat-fee programs with exact arithmetic.
- **Refund and idempotency rules** across every money operation.

- **Webhook semantics:** per-receiver de-duplication, fail-closed on a missing secret, operator re-processing.
- **Manual holds and the audit entries** behind them.
- **Convergence and reconciliation jobs:** every hourly and daily job was driven by hand from a recoverable state.
- **SDK v5** end to end, including the generated core and the end-customer readback after every money state change.
- **Console and Code Builder:** the console's tests, type checks and build; the code generator's tests; the generated-integration execution suite in containers.
- **Clean-environment authentication dependencies:** the production sign-in verification libraries install and work from the locked requirements in a fresh environment.
- **The complete automated suite** across the platform, the SDK, the contract tests and the generator.

Suite	Result
Platform, API, contracts, SDK and generator tests	8,189 passed of 8,192 collected; the 3 remaining are opt-in tests that need a real Stripe key
Console unit tests	1,342 passed
Code generator tests	902 passed
Generated-integration execution (containers)	80 passed

Outcome recorded by the verification:

`BILLING BLOCKER: 0`

`PRE-ASH REQUIRED FIX: 0`

What was deliberately not claimed. No real Stripe transport was used: no test-mode payment, checkout, invoice or webhook was sent to or received from Stripe. No production infrastructure, public endpoint, hosting, login environment or service-level measurement was involved. The verification proves the billing logic; it does not prove the deployment.

10. What is NOT built, and what is built but not yet activated

Not built or deliberately unsupported in version 1

These are recorded decisions, not oversights:

- **Foreign exchange.** US dollars only; any other currency is refused.
- **A TypeScript or JavaScript SDK.** Python only; other languages use the REST API or generated shell code.
- **A customer list or search API.** Customers are addressed by their identifier.
- **A customer "closing" workflow.** Customers are active or suspended; closure is not a supported transition.
- **Rebinding a subscription mirror** to a different customer. Intentionally unsupported; a future need would be a separately designed operator workflow.

- **Automatic reversal when a Stripe refund later fails.** UBB alerts a human, who verifies in Stripe and re-credits by hand if warranted.
- **An "all test-mode" Stripe layout.** The platform assumes one live platform account plus a test key for sandbox tenants.
- **Resolving credits and debits by the client's own customer identifier.** They take UBB's customer identifier.
- **An embeddable end-customer widget.** The end-customer readback is an API the tenant renders itself. *(Interpretation from the code inventory.)*
- **A delayed or asynchronous ingest path.** Removed on purpose; every usage call is answered synchronously.
- **Chaos or failure drills** of the deployed system. Recorded as an accepted limitation and a first post-launch hardening item.

Built but not yet production-activated

These exist, are tested and were verified locally, and are waiting on real-world readiness rather than on engineering:

- Prepaid wallets with Stripe top-ups and auto-top-up.
- Postpaid monthly usage invoices pushed to Stripe, and UBB's own platform-fee invoice.
- Fee-bearing plans and Stripe subscriptions, seats and lifecycle operations.
- Both inbound Stripe webhook receivers.
- Stripe Connect onboarding of a tenant's connected account.
- Referrals (built and verified; simply not enabled for the first client).

Each money-moving item additionally requires the human payment-rail activation described in section 4.

11. What remains before a real launch

This is external and operational launch work. It is not a hidden backlog of known billing-engine defects. The engineering handoff records zero known billing blockers; what follows is the work of putting a verified system into the world.

Area	What is needed
Hosting and deployment	Choose the hosting provider, region and account; build the one immutable application image; deploy it by digest as three roles: web, background worker and scheduler
Data stores	Managed PostgreSQL 16 and Redis 7 in one region, with the one-off database migration run as an explicit release step
Logging and alerts	A log drain for the structured application logs, and a named, accountable destination for the operational alerts UBB raises
Stripe test environment	Real Stripe test credentials, a Connect test account and a public test receiver, so the money paths can be exercised against Stripe for the first time
Stripe Connect	Onboarding of a real connected account and confirmation that it is charge-ready

Area	What is needed
Public webhook endpoints	Public HTTPS addresses for both Stripe receivers, registered with Stripe, each with its own signing secret
Production sign-in	A production Clerk environment and its keys for console member login
Service-level measurement	The three agreed service-level objectives measured on the real host, not on a laptop
Repository protection	Native required status checks on the main branch, once repository-admin access permits; the temporary merge guard used during the programme was a compensating control, not branch protection
First real tenant	Admission of the first real client in external mode, on the deployed release, with a proven client-to-UBB-and-back path
Accountable inputs	Named humans for the acceptance audit, the governance decision and the alert destination, recorded before launch
Payment rail	Activation only if and when a customer is later admitted in a money mode; not part of the first launch

The detailed launch plan exists and is explicitly marked **PRE-LAUNCH — NOT FINAL**: it cannot start until the inputs above are real and signed off.

12. Who this could be sold to, and initial positioning

This section describes positioning implied by the capabilities that exist. It is not market research and no demand has been validated.

UBB fits a business that resells metered, costly consumption and needs the economics to be exact. Based only on what is built, the natural fits are businesses that:

- meter AI model or API usage and resell it to their own customers;
- need true per-customer cost, revenue and margin rather than estimates;
- need to stop runaway spend by a customer, an agent or a job before it happens;
- want prepaid credits or postpaid monthly invoicing driven through Stripe;
- sell usage-based plans with access fees and seats;
- want their own application to receive stop and affordability signals in real time.

Example customer profiles

1. **An AI assistant product with many small customers.** Needs per-customer margin visibility and prepaid credits with automatic top-up, because supplier costs arrive per token and customers pay in advance.
2. **An agent-automation platform for businesses.** Runs long, multi-step jobs that can loop; needs task ceilings, kill switches and a clear "this job may cost at most X" promise to its customers.
3. **A developer-tools company reselling model access.** Charges monthly in arrears, needs clean postpaid invoicing through Stripe with sub-cent accuracy and payment-failure handling it does not have to write itself.
4. **A vertical SaaS adding AI features to a seat-based plan.** Already sells seats; wants to add a usage markup axis to its plans and see which features and models eat the margin.

5. **A team that has built its own metering spreadsheet.** Has outgrown ad hoc logging and needs receipts, audit trails and reconciliation before a finance or investor review.

The first real client is planned in external mode, which means the entry point to a sale does not depend on moving money through UBB on day one.

13. A page for the founder: what have we built?

What exists. A complete, tested usage-and-money control plane for AI and API businesses: metering with receipts, pricing books, tasks with ceilings, four kinds of spend control, per-customer economics, three billing modes, plans and Stripe subscriptions, referrals, a versioned API, a released Python SDK, a code generator that writes the client's integration, outbound webhooks, two inbound Stripe receivers, an audit ledger, and a tenant console covering the main surfaces. It is not a prototype. It has thousands of automated tests and was re-proven end to end after a deliberate repair programme.

What is unusually strong. The economic discipline. Every money figure is integer micro-dollars; every posting carries a sealed receipt; every money operation is idempotent; the billing owner of every invoice is frozen; the database enforces the rules the code promises. Stripe is driven, never duplicated. The stop verdict travels inside the same response as the usage record. Most products in this space bolt metering onto billing; UBB was built around the economic truth first. The spend-control story, from task ceilings to real-time customer stops, is genuinely differentiated for agentic AI workloads.

What still needs work. Nothing in the billing engine is known to be broken. What is missing is everything outside the code: hosting, real Stripe accounts and public endpoints, the production login environment, measured service levels, and the first real client. The console is capable but not polished into a self-serve product; several operations are still API or operator steps. There is only a Python SDK. Everything is US-dollar only.

What can safely be demonstrated now. On a developer machine: the full external-mode flow from configuring a tenant to recorded usage, stop signals, economics and webhooks; the prepaid and postpaid flows with Stripe data objects but no live Stripe; the console in mock mode; the SDK; the Code Builder generating and running an integration. All of this is real code doing real work; none of it is a slide.

What cannot honestly be claimed yet. That UBB is in production. That a real payment has flowed through it. That a real customer is live. That it meets any service-level target. That it has been operated under load or failure. Say "verified and awaiting launch", never "launched".

The next milestone. Deploy the verified release to real hosting, connect a real Stripe test environment, measure the service levels, and admit the first client in external mode. That milestone is defined, written down and waiting on named inputs rather than on engineering.

14. Appendix — exact release identities

For engineers who need to confirm they are looking at the same system this brief describes. These are the only places in this document where programme history appears.

Identity	Value
Current repository `main`	`90d008b709a86ff0d63588d8dd6b0a160978230b`
Behavioural proof anchor (the commit the verification ran on)	`a6738e78e96ab10eabd45dc10fafdc842b7de519`
SDK release commit	`b41b395f59d798cec83289d83d5ec71f83201c7f`
SDK release tag	`sdk-v5.0.0`
SDK wheel	`ubb_sdk-5.0.0-py3-none-any.whl`
SDK wheel SHA-256	`56d28b90876beff35532e48df6a5fb8545f665f9c781b40d9febf0b418555948`
Launch plan status	`PRE-LAUNCH — NOT FINAL`

The two commits between the proof anchor and the current `main` are documentation and release-record changes only; they changed no product behaviour. The authoritative engineering handoff from which this brief is derived is the `R10-ASH-HANDOFF-2` package; the SDK release evidence is the `R10-SDK-V5-RELEASE` package.